MySQL Triggers
1. What is a Trigger?

A trigger in MySQL is a database object that automatically executes a predefined SQL statement when a specified event occurs on a table.

In simple words:

Trigger = Automatically execute an action when INSERT, UPDATE, or DELETE happens.

For example, whenever an employee's salary is updated, we can automatically store the old salary and new salary in an audit table.

2. Syntax of a Trigger

```
DELIMITER //

CREATE TRIGGER trigger_name
BEFORE | AFTER INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
-- SQL statements
END //

DELIMITER ;
```

Important parts

Part	Meaning
CREATE TRIGGER	Creates a trigger
BEFORE	Runs before the event
AFTER	Runs after the event
INSERT	Trigger fires when a row is inserted
UPDATE	Trigger fires when a row is updated
DELETE	Trigger fires when a row is deleted
ON table_name	Specifies the table
FOR EACH ROW	Executes once for every affected row
BEGIN ... END	Contains multiple SQL statements

3. Types of Triggers

MySQL triggers are mainly classified by event and timing.

Timing
BEFORE
AFTER

Events
INSERT
UPDATE
DELETE

So there can be:

BEFORE INSERT
AFTER INSERT

BEFORE UPDATE
AFTER UPDATE

BEFORE DELETE
AFTER DELETE

4. BEFORE INSERT Trigger

A BEFORE INSERT trigger runs before a new row is inserted.

Example

Suppose we have:

```
CREATE TABLE employees (
id INT PRIMARY KEY,
name VARCHAR(100),
salary DECIMAL(10,2)
);
```

We want to make sure salary cannot be negative.

```
DELIMITER //

CREATE TRIGGER before_employee_insert
BEFORE INSERT
ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < 0 THEN
SET NEW.salary = 0;
END IF;
END //

DELIMITER ;
```

Now:

```
INSERT INTO employees
VALUES (1, 'Rahul', -5000);
```

The trigger changes the salary to:

0

NEW

For an INSERT, NEW represents the new row being inserted.

```
NEW.salary
NEW.name
NEW.id
```

5. AFTER INSERT Trigger

An AFTER INSERT trigger executes after a row has been successfully inserted.

A common use is maintaining an audit/log table.

```
Create log table
CREATE TABLE employee_log (
log_id INT AUTO_INCREMENT PRIMARY KEY,
employee_id INT,
```

```
action VARCHAR(50),
log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

Create trigger
DELIMITER //

CREATE TRIGGER after_employee_insert
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
INSERT INTO employee_log(employee_id, action)
VALUES (NEW.id, 'Employee Inserted');
END //

DELIMITER ;
```

Now:

```
INSERT INTO employees
VALUES (2, 'Amit', 30000);
```

Automatically, the following is added to employee_log:

```
employee_id | action
------------|------------------
2 | Employee Inserted
```

6. BEFORE UPDATE Trigger

A BEFORE UPDATE trigger executes before an existing row is updated.

Example: Prevent salary reduction

```
DELIMITER //

CREATE TRIGGER before_employee_update
BEFORE UPDATE
ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < OLD.salary THEN
SET NEW.salary = OLD.salary;
END IF;
END //

DELIMITER ;
```

Suppose:

Old salary = 50000
New salary = 40000

The trigger changes the new salary back to:

50000

OLD and NEW

For an UPDATE:

OLD.salary

means the salary before the update.

NEW.salary

means the salary after the update.

7. AFTER UPDATE Trigger

This is commonly used for audit/history tracking.

Suppose we create:

```
CREATE TABLE salary_history (
history_id INT AUTO_INCREMENT PRIMARY KEY,
employee_id INT,
old_salary DECIMAL(10,2),
new_salary DECIMAL(10,2),
changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Now create the trigger:

```
DELIMITER //

CREATE TRIGGER after_salary_update
AFTER UPDATE
ON employees
FOR EACH ROW
BEGIN
IF OLD.salary <> NEW.salary THEN
INSERT INTO salary_history
(employee_id, old_salary, new_salary)
VALUES
(OLD.id, OLD.salary, NEW.salary);
END IF;
END //

DELIMITER ;
```

Now execute:

```
UPDATE employees
SET salary = 60000
WHERE id = 1;
```

The trigger automatically stores:

```
employee_id | old_salary | new_salary
------------|------------|-----------
```

1 | 50000 | 60000

This is a very common real-world trigger example.

8. BEFORE DELETE Trigger

A BEFORE DELETE trigger executes before a row is deleted.

One use is preventing deletion under certain conditions.

Example

Suppose employees with salary greater than 100000 should not be deleted.

```
DELIMITER //

CREATE TRIGGER before_employee_delete
BEFORE DELETE
ON employees
FOR EACH ROW
BEGIN
IF OLD.salary > 100000 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'High-salary employees cannot be deleted';
END IF;
END //

DELIMITER ;
```

Now:

```
DELETE FROM employees
WHERE id = 5;
```

If that employee's salary is greater than 100000, MySQL generates an error instead of deleting the row.

9. AFTER DELETE Trigger

An AFTER DELETE trigger runs after a row has been deleted.

This is useful for maintaining a deleted-record history.

```
Archive table
CREATE TABLE deleted_employees (
id INT,
name VARCHAR(100),
salary DECIMAL(10,2),
deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

Trigger
DELIMITER //

CREATE TRIGGER after_employee_delete
AFTER DELETE
ON employees
FOR EACH ROW
```

```
BEGIN
INSERT INTO deleted_employees
(id, name, salary)
VALUES
(OLD.id, OLD.name, OLD.salary);
END //

DELIMITER ;
```

Now:

```
DELETE FROM employees
WHERE id = 2;
```

The deleted employee is automatically copied into:

deleted_employees

10. OLD vs NEW

This is one of the most important concepts in triggers.

Event OLD NEW
INSERT ✗ ✓
UPDATE ✓ ✓
DELETE ✓ ✗
INSERT
NEW.name
NEW.salary

UPDATE
OLD.salary
NEW.salary

DELETE
OLD.name
OLD.salary

Think of it like:

OLD = Previous value
NEW = New value

11. Why do we use DELIMITER?

Normally MySQL uses ; to identify the end of a SQL statement.

For example:

```
SELECT * FROM employees;
```

But a trigger may contain multiple statements:

```
BEGIN
INSERT ...;
UPDATE ...;
```

END

If we use the normal delimiter, MySQL may think the trigger definition has ended at the first ;.

Therefore we temporarily change the delimiter:

DELIMITER //

Then:

CREATE TRIGGER ...
BEGIN
INSERT ...;
UPDATE ...;
END //

Finally restore it:

DELIMITER ;

Important: DELIMITER is a MySQL client command; it is not part of the trigger stored in the database.

12. Trigger With Multiple Statements

Suppose when a new employee is added, we want to:

Insert the employee.
Add an entry to the log.
DELIMITER //

CREATE TRIGGER employee_insert_log
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
INSERT INTO employee_log(employee_id, action)
VALUES (NEW.id, 'Employee Added');

INSERT INTO salary_history(employee_id, old_salary, new_salary)
VALUES (NEW.id, 0, NEW.salary);
END //

DELIMITER ;

Multiple statements are placed between:

BEGIN
...
END

13. Using IF in a Trigger

Triggers can contain conditional logic.

DELIMITER //

```
CREATE TRIGGER check_salary
BEFORE INSERT
ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < 15000 THEN
SET NEW.salary = 15000;
END IF;
END //

DELIMITER ;
```

If:

```
INSERT INTO employees
VALUES (10, 'Ravi', 10000);
```

The trigger changes it to:

15000

14. Using SIGNAL

SIGNAL allows a trigger to generate a custom error.

Example

```
DELIMITER //

CREATE TRIGGER validate_salary
BEFORE INSERT
ON employees
FOR EACH ROW
BEGIN
IF NEW.salary < 10000 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Salary must be at least 10000';
END IF;
END //

DELIMITER ;
```

Now:

```
INSERT INTO employees
VALUES (20, 'John', 5000);
```

Result:

ERROR: Salary must be at least 10000

This is useful for data validation.

15. Real-World Example: Product Stock

Consider an e-commerce system.

```
CREATE TABLE products (
product_id INT PRIMARY KEY,
product_name VARCHAR(100),
stock INT
);
```

Suppose we don't want stock to become negative.

```
DELIMITER //

CREATE TRIGGER prevent_negative_stock
BEFORE UPDATE
ON products
FOR EACH ROW
BEGIN
IF NEW.stock < 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Stock cannot be negative';
END IF;
END //

DELIMITER ;
```

Now:

```
UPDATE products
SET stock = -5
WHERE product_id = 1;
```

The trigger prevents the update.

16. Real-World Example: Order Audit

Suppose:

```
CREATE TABLE orders (
order_id INT PRIMARY KEY,
customer_name VARCHAR(100),
amount DECIMAL(10,2),
status VARCHAR(30)
);
```

Create an audit table:

```
CREATE TABLE order_audit (
audit_id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT,
old_status VARCHAR(30),
new_status VARCHAR(30),
changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Trigger:

```
DELIMITER //

CREATE TRIGGER order_status_audit
AFTER UPDATE
ON orders
FOR EACH ROW
BEGIN
IF OLD.status <> NEW.status THEN
INSERT INTO order_audit
(order_id, old_status, new_status)
VALUES
(OLD.order_id, OLD.status, NEW.status);
END IF;
END //

DELIMITER ;
```

If:

```
UPDATE orders
SET status = 'Shipped'
WHERE order_id = 101;
```

The history is automatically stored.

This is a good example of using a trigger for auditing.

17. How to See Existing Triggers

To display all triggers:

```
SHOW TRIGGERS;
```

For a particular database:

```
SHOW TRIGGERS
FROM database_name;
```

18. How to Delete a Trigger

```
DROP TRIGGER trigger_name;
```

Example:

```
DROP TRIGGER before_employee_insert;
```

Safer version:

```
DROP TRIGGER IF EXISTS before_employee_insert;
```

19. Important Rules

Rule 1: Trigger automatically executes

You don't normally call a trigger manually.

You execute:

INSERT
UPDATE
DELETE

and MySQL fires the appropriate trigger automatically.

Rule 2: FOR EACH ROW

MySQL triggers are row-level triggers.

FOR EACH ROW

means the trigger executes separately for every affected row.

For example:

```
UPDATE employees
SET salary = salary + 5000;
```

If 100 employees are updated, the trigger can execute 100 times.

Rule 3: INSERT uses NEW
NEW.column_name

Rule 4: DELETE uses OLD
OLD.column_name

Rule 5: UPDATE uses both
OLD.column_name
NEW.column_name

20. Trigger vs Stored Procedure
Trigger Stored Procedure
Automatically executes Usually called explicitly
Associated with a table event Independent database routine
Fires on INSERT/UPDATE/DELETE Can perform various operations
Cannot be called like a normal procedure Can be called using CALL
Useful for auditing/validation Useful for reusable business logic
Example

Trigger:

```
UPDATE employees
SET salary = 50000
WHERE id = 1;
```

The trigger automatically executes.

Procedure:

```
CALL update_employee_salary(1, 50000);
```

You explicitly call it.

21. Common Uses of Triggers

Triggers are commonly used for:

Auditing changes
Maintaining history tables
Data validation
Automatically updating related tables
Preventing invalid operations
Maintaining derived data
Logging INSERT/UPDATE/DELETE operations
22. Quick Revision

Remember this:

TRIGGER
|
+---------+---------+
| |
BEFORE AFTER
| |
+-----+-----+ +-----+-----+
| | | | | |
INSERT UPDATE DELETE INSERT UPDATE DELETE

And memorize:

INSERT → NEW
UPDATE → OLD + NEW
DELETE → OLD

Most important syntax
DELIMITER //

CREATE TRIGGER trigger_name
AFTER INSERT
ON table_name
FOR EACH ROW
BEGIN
-- statements
END //

DELIMITER ;

Most common practical example
DELIMITER //

CREATE TRIGGER salary_audit
AFTER UPDATE
ON employees
FOR EACH ROW
BEGIN
IF OLD.salary <> NEW.salary THEN
INSERT INTO salary_history
(employee_id, old_salary, new_salary)
VALUES
(OLD.id, OLD.salary, NEW.salary);
END IF;
END //

DELIMITER ;

Exam/interview one-liner:

A MySQL trigger is a database object that automatically executes a specified action when an INSERT, UPDATE, or DELETE event occurs on a table.